

Exercise Series
Chapter 3 – Linked Lists
Book Library — Singly Linked List

Topics Covered
*Book Node Structure • List Traversal • Insert at Beginning / Middle / End
Delete from Beginning / Middle / End • Memory Management*

Prepared by the CS Department • Spring 2026

1. Book Node Structure

Each node in the Book Linked List stores four fields describing a book, plus a pointer to the next node in the chain. Study the structure carefully before attempting any exercise.

1.1 Struct Definition

```
// — Book node definition
struct Book {
    char    title[100];        // Title of the book
    char    author[50];        // Author's full name
    int     year;              // Publication year
    float   price;             // Price in USD
    struct Book* next;         // Pointer to next book node
};
typedef struct Book Book;
```

1.2 Creating a Book Node

Use `malloc()` to allocate each new node dynamically. Always copy strings with `strcpy()`.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

Book* createBook(char* title, char* author, int year, float price) {
    Book* newBook = (Book*) malloc(sizeof(Book));
    if (newBook == NULL) { printf("Allocation failed!\n"); return NULL; }
    strcpy(newBook->title, title);
    strcpy(newBook->author, author);
    newBook->year = year;
    newBook->price = price;
    newBook->next = NULL;
    return newBook;
}
```

1.3 Printing the Book List

```
void printBookList(Book* head) {
    Book* current = head;
    int i = 1;
    while (current != NULL) {
        printf("[%d] \"%s\" by %s (%d) - $%.2f\n",
            i, current->title, current->author,
            current->year, current->price);
        current = current->next;
        i++;
    }
    printf("--- NULL ---\n");
}
```

1.4 Sample Book Catalog — use these books in all exercises

#	Title	Author	Year	Price (\$)
1	<i>Clean Code</i>	Robert C. Martin	2008	39.99
2	<i>The Pragmatic Programmer</i>	David Thomas	1999	45.00
3	<i>Introduction to Algorithms</i>	Thomas H. Cormen	2009	89.95
4	<i>Design Patterns</i>	Erich Gamma	1994	54.99
5	<i>You Don't Know JS</i>	Kyle Simpson	2015	34.99
6	<i>The Art of Computer Programming</i>	Donald Knuth	1968	120.00

Exercise 1 — Inserting Books into the Linked List

Objective: Implement the three core insertion functions for a singly linked list of books. Each function must correctly manipulate pointers and handle edge cases (empty list, invalid position).

Part A — Insert at the Beginning ($O(1)$)

Inserting at the beginning is the fastest operation — no traversal is needed. The new node becomes the new head immediately.

Algorithm: (1) Allocate new Book node. (2) Set `newBook→next = head`. (3) Return `newBook` as the new head.

Task 1.1 — `insertAtBeginning()`

Write the function: `Book* insertAtBeginning(Book* head, char* title, char* author, int year, float price)`

- Allocate a new Book node using `createBook()`.
- Set the new node's next pointer to the current head.
- Return the new node as the updated head.
- Handle the case where head is NULL (list is empty).

Expected output after running Task 1.1:

```
// Call: head = insertAtBeginning(head, "You Don't Know JS", "Kyle Simpson", 2015, 34.99);
// Then: head = insertAtBeginning(head, "Clean Code", "Robert C. Martin", 2008, 39.99);
```

```
[1] "Clean Code" by Robert C. Martin (2008) - $39.99
```

```
[2] "You Don't Know JS" by Kyle Simpson (2015) - $34.99
--- NULL ---
```

Part B — Insert at the End ($O(n)$)

To insert at the end, traverse the entire list until the last node (`next == NULL`), then link it to the new node.

Algorithm: (1) If list empty, return new node. (2) Walk to last node. (3) Set `last→next = newBook`.

Task 1.2 — `insertAtEnd()`

Write the function: `Book* insertAtEnd(Book* head, char* title, char* author, int year, float price)`

- Allocate a new Book node using `createBook()`.
- If the list is empty (`head == NULL`), return the new node directly.
- Otherwise, traverse the list using a loop until `current->next == NULL`.
- Link the last node to the new book node.
- Return the unchanged head.

Expected output after building the list with `insertAtEnd()`:

```
// Add books 1→2→3 using insertAtEnd():
[1] "Clean Code" by Robert C. Martin (2008) - $39.99
[2] "The Pragmatic Programmer" by David Thomas (1999) - $45.00
[3] "Introduction to Algorithms" by Thomas H. Cormen (2009) - $89.95
--- NULL ---
```

Part C — Insert at a Given Position ($O(n)$)

To insert at position `pos` (1-indexed), traverse to the node at position `pos - 1`, then re-link pointers to insert the new node between the predecessor and its successor.

Algorithm: (1) If `pos = 1`, call `insertAtBeginning()`. (2) Walk to node `pos-1`. (3) Set `newBook→next = prev→next`. (4) Set `prev→next = newBook`.

Task 1.3 — `insertAtPosition()`

Write the function: `Book* insertAtPosition(Book* head, char* title, char* author, int year, float price, int pos)`

- If `pos <= 1`, delegate to `insertAtBeginning()` and return.
- Walk the list with a counter until you reach node at index `pos-1`.
- If the position is out of range (`current == NULL`), print an error, free the unused node, and return head unchanged.
- Set `newBook->next = current->next` (point new node to successor).
- Set `current->next = newBook` (link predecessor to new node).
- Return the unchanged head.

Expected output — inserting at position 2 into a 3-book list:

```
// Initial list: Clean Code -> Introduction to Algorithms -> Design Patterns -> NULL
// Call: head = insertAtPosition(head, "The Pragmatic Programmer", "David Thomas", 1999,
45.00, 2);

[1] "Clean Code" by Robert C. Martin (2008) - $39.99
[2] "The Pragmatic Programmer" by David Thomas (1999) - $45.00
[3] "Introduction to Algorithms" by Thomas H. Cormen (2009) - $89.95
[4] "Design Patterns" by Erich Gamma (1994) - $54.99
--- NULL ---
```

Part D — Complete main() Program

Task 1.4 — Build and Verify the Full List

In main(), build a list of all 6 books from the catalog using a mix of the three insertion functions. Print the list after each insertion to verify pointer correctness.

- Use insertAtEnd() to add: Clean Code, Introduction to Algorithms, Design Patterns.
- Use insertAtBeginning() to add: You Don't Know JS (it should appear first).
- Use insertAtPosition() to insert: The Pragmatic Programmer at position 2.
- Use insertAtEnd() to add: The Art of Computer Programming at the end.
- Print the list after each step — verify the order is correct.
- At the end of main(), free every node using a helper freeBookList() function.

Hints & Reminders

- Always use strcpy() — never assign strings directly with =.
- After each insertion, call printBookList() to trace the state of the list.
- For insertAtPosition(), test with pos = 1 (should behave like insertAtBeginning).
- Test with pos larger than the list length — verify the out-of-range message appears.
- Write freeBookList(Book* head) that walks and frees every node; call it at end of main().

Exercise 2 — Deleting Books from the Linked List

Objective: Implement the three core deletion functions for the Book Linked List. You must correctly re-link predecessor pointers and release memory for every deleted node.

General strategy: (1) Locate the node to delete and track its predecessor. (2) Re-link predecessor→next to skip the target. (3) Call free() to release the node.

Part A — Delete from the Beginning (O(1))

The simplest deletion: advance head to the second node, then free the original first node.

Task 2.1 — deleteFromBeginning()

Write the function: Book* deleteFromBeginning(Book* head)

- If head == NULL, print "List is empty." and return NULL.
- Save a temporary pointer to the current head node.
- Advance head to head->next (the second node becomes the new head).
- Call free() on the saved pointer to release the old head.
- Print the title of the deleted book before freeing.
- Return the updated head.

Expected output:

```
// Before: Clean Code -> The Pragmatic Programmer -> Introduction to Algorithms -> NULL
// Call: head = deleteFromBeginning(head);
```

Deleted: "Clean Code"

```
[1] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
```

```
[2] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Part B — Delete from the End ($O(n)$)

Traverse to the second-to-last node, set its next to NULL, then free the last node.

Edge cases: (1) Empty list → return NULL. (2) List with exactly one node → free it, return NULL.

Task 2.2 — deleteFromEnd()

Write the function: `Book* deleteFromEnd(Book* head)`

- If `head == NULL`, print "List is empty." and return NULL.
- If `head->next == NULL` (single node), print its title, free it, and return NULL.
- Otherwise, traverse with current until `current->next->next == NULL` (second-to-last node).
- Print the title of `current->next` before freeing.
- Free `current->next` (the last node).
- Set `current->next = NULL` to mark the new end.
- Return the unchanged head.

Expected output:

```
// Before: The Pragmatic Programmer -> Introduction to Algorithms -> Design Patterns ->
NULL
// Call: head = deleteFromEnd(head);

Deleted: "Design Patterns"
[1] "The Pragmatic Programmer" by David Thomas (1999) — $45.00
[2] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Part C — Delete by Title (Middle / Any Position) ($O(n)$)

The general deletion: locate the first node whose title matches a target string using `strcmp()`, re-link pointers around it, and free it.

Two sub-cases: (1) The matching node is the head. (2) The matching node is somewhere in the middle or at the end.

Task 2.3 — deleteByTitle()

Write the function: `Book* deleteByTitle(Book* head, char* targetTitle)`

- If `head == NULL`, print "List is empty." and return NULL.
- If `strcmp(head->title, targetTitle) == 0` → the head is the target: save it, advance head, free the saved node, return new head.
- Otherwise, traverse with current until `current->next != NULL`:
 - If `strcmp(current->next->title, targetTitle) == 0`: save `current->next`, set `current->next = saved->next`, free saved, return head.
- If the loop ends without a match, print: "Book '[title]' not found in the list."
- Return head unchanged if no match found.

Expected output — deleting a middle node:

```
// Before: Clean Code -> The Pragmatic Programmer -> Introduction to Algorithms -> NULL
// Call: head = deleteByTitle(head, "The Pragmatic Programmer");

Deleted: "The Pragmatic Programmer"
```

```
[1] "Clean Code" by Robert C. Martin (2008) — $39.99
[2] "Introduction to Algorithms" by Thomas H. Cormen (2009) — $89.95
--- NULL ---
```

Part D — Complete main() with All Deletions

➡ Task 2.4 — Full Deletion Sequence

Build the 6-book list from Exercise 1, then perform all three deletion operations in order. Print the list after each deletion.

- Build initial list (all 6 books) — reuse your insertion functions from Exercise 1.
- Call `deleteFromBeginning(head)` — removes the first book.
- Print the list. Call `deleteByTitle(head, "Design Patterns")` — removes a middle book.
- Print the list. Call `deleteFromEnd(head)` — removes the last remaining book.
- Print the final list.
- Free all remaining nodes using `freeBookList()` — check for memory leaks.

Bonus Challenge — Advanced Book List Operations

These tasks go beyond standard insertion and deletion. Attempt them only after completing Exercises 1 and 2.

➡ Bonus Task A — `deleteMostExpensive()`

Write: `Book* deleteMostExpensive(Book* head)` Find the book with the highest price and remove it from the list. If multiple books share the same maximum price, remove the first one found.

- First pass: traverse the list to find the maximum price.
- Second pass (or combine with first): locate and delete the node with that price.
- Test with the full 6-book catalog — verify that "The Art of Computer Programming" (\$120.00) is removed.

➡ Bonus Task B — `deleteByYear()`

Write: `Book* deleteByYear(Book* head, int year)` Remove ALL books published in the given year. Multiple deletions may occur in a single call.

- Iterate through the entire list — multiple books may share the same year.
- Handle consecutive matching nodes carefully (update head if needed).
- Test: `deleteByYear(head, 2008)` should remove "Clean Code" (2008) and keep all others.

➡ Bonus Task C — `insertSorted()`

Write: `Book* insertSorted(Book* head, char* title, char* author, int year, float price)` Insert a new book into the list so that books remain sorted alphabetically by title (A→Z).

- Use `strcmp()` to compare titles while traversing.
- Find the correct position where the new book's title comes after the predecessor and before the successor.

- Test by inserting several books out of order and verifying the sorted output.

Summary Operation Reference — Book Linked List

Operation	Time	Key Action
<code>insertAtBeginning()</code>	$O(1)$	<code>newBook->next = head</code> ; return <code>newBook</code>
<code>insertAtEnd()</code>	$O(n)$	Walk to last node; <code>last->next = newBook</code>
<code>insertAtPosition()</code>	$O(n)$	Walk to <code>pos-1</code> ; relink around new node
<code>deleteFromBeginning()</code>	$O(1)$	<code>head = head->next</code> ; free old head
<code>deleteFromEnd()</code>	$O(n)$	Walk to 2nd-to-last; free last, set NULL
<code>deleteByTitle()</code>	$O(n)$	Find predecessor; relink; free target
<code>printBookList()</code>	$O(n)$	Follow next pointers, print each book

Key Takeaways

- Always use `strcpy()` for string fields — direct assignment with `=` will not copy the string.
- Call `free()` for every deleted node — one deletion = one `free()`.
- Check for `head == NULL` at the start of every function before dereferencing any pointer.
- For middle deletion, always keep track of the predecessor node using a `prev` or trailing pointer.
- Use `valgrind ./program` (if available) to confirm there are zero memory leaks.